Advances in Cognitive Systems 13 (2026) 1-6 Submitted 8/2026; published 10/2026

Planning With Linguistic Hierarchical Task Networks

Glen Smith GLENSMITH@GATECH.EDU
Christopher J. MacLellan CMACLELL@GATECH.EDU
College of Computing, Georgia Institute of Technology, Atlanta, GA 30332 USA

Abstract

Hierarchical Task Networks (HTNs) provide structured representations of procedural knowledge, but their reliance on formal symbolic representations can limit reuse when the same procedural task is expressed in other environment representations. We introduce Linguistic HTN (L-HTN) planning, which represents tasks, methods, and operators in natural language and uses a large language model to interpret environment observations, evaluate method preconditions, and ground task arguments during planning. The planning process is constrained to the L-HTN's procedural structure while the language model performs flexible pattern matching with local semantic interpretation to decompose tasks. We evaluate our approach on two intelligent tutoring tasks, measuring whether unmodified L-HTNs can remain performant across controlled changes to the environment's representation. We also compare L-HTNs to traditional HTNs and unconstrained LLMs in each case. We find that while HTNs and unconstrained LLMs exhibit highly degraded performance for many representations, L-HTNs are able to flexibly apply their task knowledge to remain robust across variations.

1. Introduction

Hierarchical Task Networks (HTNs) represent procedural knowledge as methods that hierarchically decompose task and primitive operators that act on an environment (Nau et al., 2003; Nejati et al., 2006). Traditional HTNs express this knowledge using formal symbolic representations such as logical predicates. These representations support precise planning, but they require HTN knowledge that uses vocabulary and relational structure that aligns and ground with the environment. For example, an HTN authored to execute a procedure in a specific web application may fail if a target input's identifier changes from `user_name` to `user_id`, or if changes to the page layout invalidate a relation such as $contains(element_1, element_2)$. In these kinds of situations, HTNs must be modified before they can be applied in the new environment.

Recent work has used large language models (LLMs) and vision-language models (VLMs) to solve procedural problems by allowing the model itself to interpret goals, choose among available actions, and construct a solution through successive interactions with the environment Yao et al. (2022). Because these models can relate natural-language instructions to previously unseen states and action descriptions, they can respond to variations that would require explicit changes to a traditional symbolic representation. However, unlike traditional cognitive systems, LLM agents do not rely on an explicit procedural model that constrains how they solve a task. Without this structure, they can produce invalid task decompositions, violate domain constraints, or generate action sequences that the environment cannot execute (Valmeekam et al., 2022). Limited context windows can also

© 2026 Cognitive Systems Foundation. All rights reserved.

cause an LLM to lose relevant information or attend to the wrong information as a long procedure progresses. Empirical studies of hierarchical planning similarly find that standalone LLMs can struggle to construct valid hierarchical decompositions.

Recent systems have combined LLMs with mechanisms for task decomposition, search, feedback, and verification rather than relying on the LLM to control the planning process. These systems occupy a broader design space in which language models perform particular functions within more structured cognitive systems. Some use language models primarily during learning to generate or augment missing task knowledge (Kirk et al., 2024; Lawley & Maclellan, 2024; Smith & MacLellan, 2025), while others invoke them during planning to acquire or repair knowledge needed for the current problem. For example, ChatHTN retains a symbolic planner but queries an LLM for task decompositions when its existing methods do not apply, then uses those decompositions to learn generalized HTN methods (Munoz-Avila et al., 2025; Xu & Munoz-Avila, 2025). HARMONIC similarly retains explicit control over the problem-solving process while restricting the language model to particular perceptual or semantic roles (Oruganti et al., 2026).

These systems show several ways to combine explicit cognitive structure with the ability of language models to flexibly interpret representations. We explore a different configuration by placing an explicit HTN architecture on an LLM substrate. Rather than using the language model only to supply missing knowledge to an otherwise symbolic planner, the planner uses the LLM throughout decomposition to relate authored procedural knowledge to the current environment. The authored HTN still determines the procedural structure, while the LLM interprets descriptions that do not need to match the environment exactly. This design targets the ability to preserve explicit procedural constraints while grounding and applying the same authored knowledge across different representations of a task environment.

In this paper, we introduce Linguistic HTN planning, which represents the core components of HTNs in English and uses an LLM to interpret those descriptions in relation to text- or image-based representations of the environment. L-HTNs retain core commitments to hierarchical task decompositions based on on methods and primitive operators analogous to those used in traditional HTNs. However, in L-HTNs, authors describe components such as method preconditions in natural language rather than in formal symbolic predicates. L-HTNs follow the typical HTN decomposition process, which constrains the LLM to valid task decompositions. We enforce structure constraints using native LLM capabilities, including callable tools and structured outputs. The LLM interprets state and task descriptions and determines which methods apply, while the planner retains explicit control over the procedural structure.

We evaluate the planner in two studies across two distinct web-based environments, an intelligent tutor for fraction arithmetic and the Prisoners and Guards logic puzzle[1]. Both domains require the planner to carry out multi-step procedures and choose among multiple methods during decomposition. Their web interfaces also let us systematically change how the same underlying task is presented without changing the task itself. Because relevant state information is represented visually in both environments, the domains also support a comparison of planning with and without visual observations.

1. Also known as "Missionaries and Cannibals".

Study 1 tests whether the planner can use L-HTNs to solve the baseline version of each task. Study 2 holds the underlying task and authored L-HTN fixed while systematically changing how the environment represents that task. Within Study 2, we compare planning with and without visual observations to test whether access to the interface image helps the LLM relate the authored procedure to the current environment. We structure the two studies this way to separate the planner's ability to follow an authored procedure from its ability to reuse that procedure across different representations of the same task.

This paper makes three claims. First, linguistic HTN planning can preserve explicit hierarchical task structure while using a language model to interpret natural-language task descriptions, method applicability conditions, and environment observations. Second, this structure allows the planner to execute plans more successfully relative to a comparable LLM agent that has access to the same primitive actions but no authored HTN. Third, by using a language model to interpret the HTN knowledge rather than requiring exact symbolic matches, the planner can reuse the same authored HTN across changes to an environment while the underlying task remains unchanged. We evaluate the second and third claims against LLM-only and symbolic HTN baselines across two web-based tutoring domains and controlled environment variations. In the following sections, we describe the linguistic HTN representation and planning approach that support these claims before presenting the empirical evaluation.

2. Background

2.1 Hierarchical Task Networks

2.1.1 Definition

Hierarchical Task Networks (HTNs) represent procedural knowledge as a hierarchy of task decompositions (Nau et al., 2003).

An HTN domain is defined as

$$\mathcal{D} = (\mathcal{T}, \mathcal{M}, \mathcal{O}, \mathcal{A}),$$

where $\mathcal{T}$ is the set of tasks definitions, $\mathcal{M}$ is the set of methods that decompose those tasks, $\mathcal{O}$ is the set of primitive operators that describe actions an agent can take, and $\mathcal{A}$ refers to a set of axioms that can generally be used as heuristics to determine method or operator applicability during planning. We do not discuss axioms in this work.

An operator $o \in \mathcal{O}$ is commonly represented as

$$o = \big(\mathrm{head}(o), \mathrm{pre}(o), \mathrm{eff}(o)\big),$$

where $\mathrm{head}(o)$ identifies the primitive task and its arguments, $\mathrm{pre}(o)$, or the *preconditions*, specifies the conditions under which the operator can be executed, and $\mathrm{eff}(o)$, or *effects*, describe the results of the operator executions. In *reactive planning* (Georgeff & Lansky, 1987; Langley et al., 1994), the planner iteratively produces single-operator plans in response to the environment state and operators may act directly on an external environment.

A method $m \in \mathcal{M}$ defines one way to accomplish a compound task and is represented as

$$m = \big(\mathrm{head}(m), \mathrm{pre}(m), \mathrm{sub}(m)\big),$$

where $\mathrm{head}(m)$ identifies the compound task and its arguments, $\mathrm{pre}(m)$ specifies when the method is applicable, and $\mathrm{sub}(m)$ is the set of subtasks that decompose the parent task.

2.1.2 Planning

A planning problem $P = (\mathcal{D}, T_0, s_0)$ consists of an HTN domain $\mathcal{D}$, an initial task set T_0, and an initial environment state s_0. A reactive planner selects a method whose preconditions are satisfied in the current state and recursively decomposes its subtasks until it reaches primitive operators. The planner executes these operators in the environment or stores their results for internal use, then observes the resulting state before continuing. Once the planner completes all of a method's subtasks, it returns to the parent method and continues with the next subtask. If a decomposition fails because an operator cannot execute or no method applies in the current state, the planner backtracks to the parent task and considers another applicable method. This process continues until all initial tasks are complete.

2.2 Language Models

Large language models (LLMs) are neural models trained on large collections of textual data (Vaswani et al., 2017). Through next-token prediction, they learn distributed representations of semantic and syntactic regularities that support text generation and the interpretation of previously unseen text (Zhao et al., 2023).

In a planning context, LLMs can be used to generate plans, but they do not inherently enforce the constraints of a planning problem. As a consequence, a generated plan may violate domain constraints, suggest methods and operators that do not apply, or simply fail to complete the stated task (Valmeekam et al., 2022). Additionally, as the planning history grows, limited context windows could make older information inaccessible, making it more difficult for the model to maintain a coherent plan over long horizons. These limitations have led to recent work that embeds an LLM within a structured planning architecture. This allows the system to assign tasks such as text interpretation to the language model while using an explicit task structure to constrain the available decompositions and actions (Kambhampati et al., 2024; Munoz-Avila et al., 2025; Oruganti et al., 2026).

3. Linguistic HTNs

3.1 Representation

A linguistic HTN (L-HTN) contains the same basic elements as a traditional HTN, but represents its tasks, methods, and operators in natural language rather than with formal predicates. Each component has a name, an English description, and a set of typed arguments with descriptions. For example, the Prisoners and Guards domain contains a task named *move prisoners to the right bank*, whose decomposition moves two prisoners from one bank to another. Its arguments specify the prisoners to move and the destination bank. Methods specify an ordered set of subtasks, whereas operators effects are implemented using executable functions that generate intermediate values or responses for

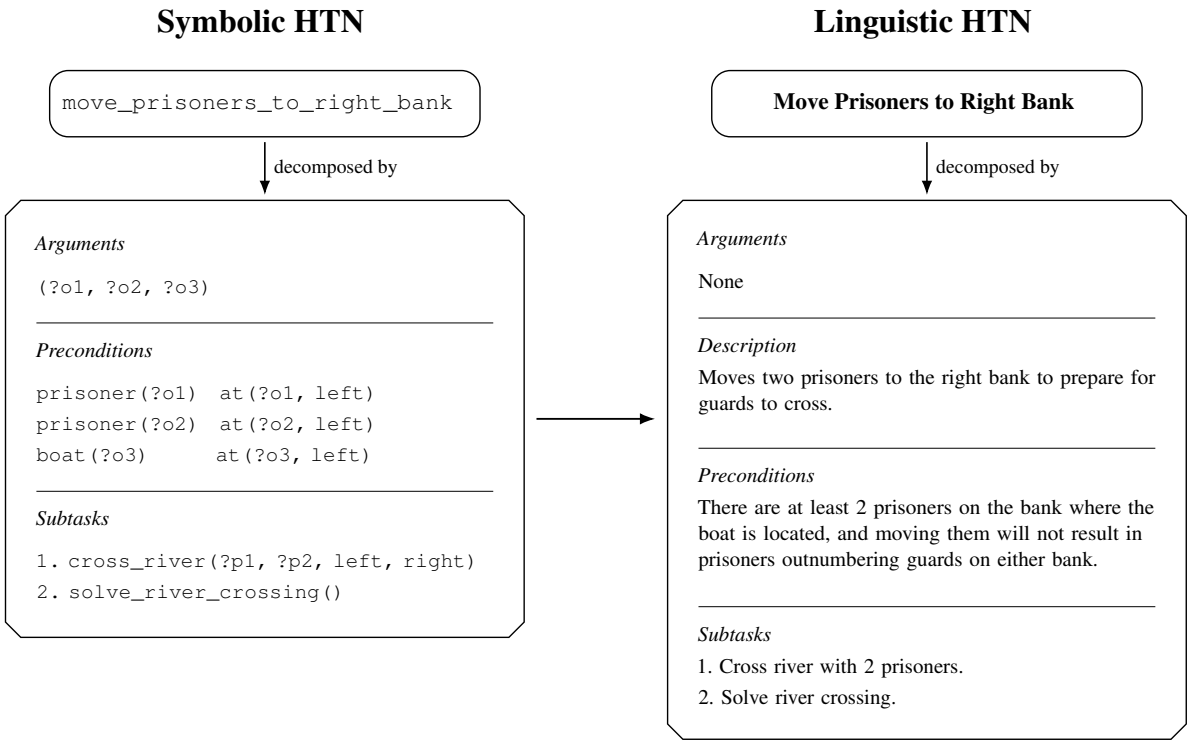


Figure 1: Comparison of a symbolic HTN method decomposition and the corresponding linguistic HTN decomposition for moving prisoners to the right bank. Note here that we did not define arguments for this L-HTN method. Instead, the LLM binds to the prisoner objects when instantiating the first 'cross the river' subtask.

the environment. Preconditions within methods and operators are also described in natural-language. Figure 1 shows how the same HTN knowledge is represented in both a traditional symbolic HTN and an L-HTN.

3.1.1 LLM Function Tools

Function tools provide the structured interface that the planner uses to obtain grounded task specifications from the language model. Each tool has a name, description, and parameter schema (See Appendix C). When the model selects a tool, it returns a structured function call containing the tool name and values for its arguments. The model does not execute the corresponding operation. The planner interprets the call and either decomposes the selected task further or executes its associated operator when the task is primitive.

The Linguistic HTN Planner maps each authored task directly onto a function tool. The task name determines the function name, its linguistic description becomes the function description, and its arguments become properties in the parameter schema. For example, the Cross the River task includes arguments for guards, prisoners, source, and destination. The schema can restrict arguments

to values allowed by the domain. The source and destination arguments can be limited to **left** and **right**, while Boolean arguments can be limited to **true** and **false**. In general, function tools guarantee the LLM output structure and the type or membership of an argument in a valid set. By directly mapping domain tasks to function names, they also guarantee that the returned functions correspond to valid tasks.

3.2 Planning with Linguistic HTNs

The planner solves a problem by recursively decomposing tasks in a depth-first fashion. At each step, the HTN algorithm maintains the global procedural context, while the language model interprets the current task, relevant methods, and current environment state. The state can be provided as text, an image, or both, and the model uses these observations to evaluate method preconditions and instantiate the next subtask for decomposition.

Although we express L-HTN knowledge in English, the planner instantiates tasks during planning in a form similar to task predicates in a traditional symbolic HTN. The language model selects a task and grounds its arguments by returning a structured function call. A call such as $cross_the_river(p1, p2, left, right)$ identifies both the selected task and the values bound to its arguments.

The planner asks the LLM to make only one local decision at a time rather than generate a complete plan. It verifies each returned decision before incorporating it into the active plan and uses retries and backtracking to recover from some incorrect choices. Unlike exact symbolic matching, however, evaluating whether linguistic preconditions hold remains a language-model judgment and is not independently verified by the planner.

3.2.1 User Request Segmentation

Planning begins by querying the LLM to map the user's request onto one or more tasks defined in the HTN. As done in systems like (Lawley & Maclellan, 2024), segmenting the initial request allows the planner to extract all of the commands from the request that the system can perform. The segmentation query provides the initial task request, current environment state, and function tools for all domain tasks. The LLM interprets the request, determines whether it corresponds to one task or several tasks, and returns the selected tasks as grounded function calls. These calls become the initial task queue. If the LLM returns no function calls, the planner allows up to three attempts before terminating.

3.2.2 Task Decomposition

When the current task is compound, the planner provides the LLM with the grounded task and its candidate methods, the current environment state, and function tools for all domain tasks. The model evaluates each method's linguistic preconditions in order to select a method and what is returned is the grounded, first subtask of that method as a function call. Decomposition continues until the planner reaches a primitive task with an executable operator. The planner executes the operator and records its result, which may be an environment action result or a value produced internally. Successful results can be supplied to the LLM when grounding later subtasks. If execution fails, the

planner backtracks to the parent task, marks the selected method as attempted, and asks the model to choose among the remaining methods.

After the first subtask of a method succeeds, the planner queries the model to instantiate the next subtask in the selected method. In this case, because the LLM does not need to select a method, we ask it to simply instantiate the next subtask in the list. In addition to the other information provided in the prompt, here the LLM also receives all results from previous subtasks. Throughout planning, the context the LLM receives is localized to the current method being solved. This allows the system to plan over recursive or potentially infinite decompositions. Planning ends when all initial tasks have completed or backtracking exhausts the available methods.

4. Experiments

4.1 Task Environments

We evaluated the planner in two web-based tutoring environments, Fraction Arithmetic and Prisoners and Guards. The Fraction Arithmetic tutor supports six problem types: addition with equal denominators, addition with unequal denominators, subtraction with equal denominators, subtraction with unequal denominators, multiplication, and division. The applicable procedure depends on the arithmetic operation and, for addition and subtraction, the relationship between the denominators. The Prisoners and Guards tutor requires the agent to move three guards and three prisoners across a river while ensuring that prisoners never outnumber guards on either bank unless no guards are present. Solving the puzzle requires a sequence of forward and return trips based on the current distribution of people and the location of the boat.

Observations from both environments are primarily represented by their HTML structure, which includes all elements contained within a tutor canvas area. In Study 2, some of the systems receive a PNG image of the tutor interface, which supplements the HTML with visual and spatial information.

4.2 Model Comparisons

We compare the Linguistic HTN Planner with an LLM-only agent and a traditional symbolic HTN planner. These comparisons help determine whether we can attribute the L-HTN's performance to its architecture rather than from the underlying language model or hierarchical task structure alone.

4.3 Study 1: Baseline Planning Performance

4.3.1 Design and Procedure

Study 1 compared the three architectures in the canonical version of each tutor. The Fraction Arithmetic evaluation contained 15 problems for each of six problem types, including addition with equal denominators, addition with unequal denominators, subtraction with equal denominators, subtraction with unequal denominators, multiplication, and division. This produced 90 problems per run. Each architecture completed the full problem set across five runs, producing 450 Fraction Arithmetic planning sessions per architecture. For Prisoners and Guards, each architecture attempted the canonical three-guard, three-prisoner problem 50 times. Study 1 therefore contained 500 independent planning

Condition	Manipulation	
	Fraction Arithmetic	**Prisoners and Guards**
Baseline	Original labels, DOM structure, and layout.	Original labels, DOM structure, and layout.
Lexical	Renamed element IDs, classes, and text to generic math terminology (e.g., changing ID `num1` to `top_number` and the title to "Solve the Expression").	Changed the theme from guards and prisoners to dogs and cats, renaming IDs, text labels (e.g., `G1` to `D1`), and CSS classes (e.g., `.boat` to `.ferry`).
Spatial	Reversed the visual ordering of the operands so that the right fraction appears before the left fraction and put the convert fraction area on top. DOM is unchanged.	Rotated the workspace $90°$ so banks appear top/bottom and the boat moves vertically. DOM is unchanged.
Structural	Completely unnested and flattened all elements into direct siblings.	Completely unnested and flattened all workspace elements (banks, boat, characters) into direct siblings.
Noise	Injected distracting functional elements, including a worked example area and a common denominator scratchpad with extra input fields.	Injected plausible but non-interactable distracting entities directly onto the river banks, such as 'wardens' (`W1`), 'spectators', and 'cargo' crates.
Reduced Info.	Stripped all text labels from the interface. Replaced element IDs in DOM with non-semantic, enumerated identifiers (e.g., `num1` becomes `x1`).	Same as fraction arithmetic.

Table 1: Specific, controlled environment changes across the two tutoring domains for Study 2.

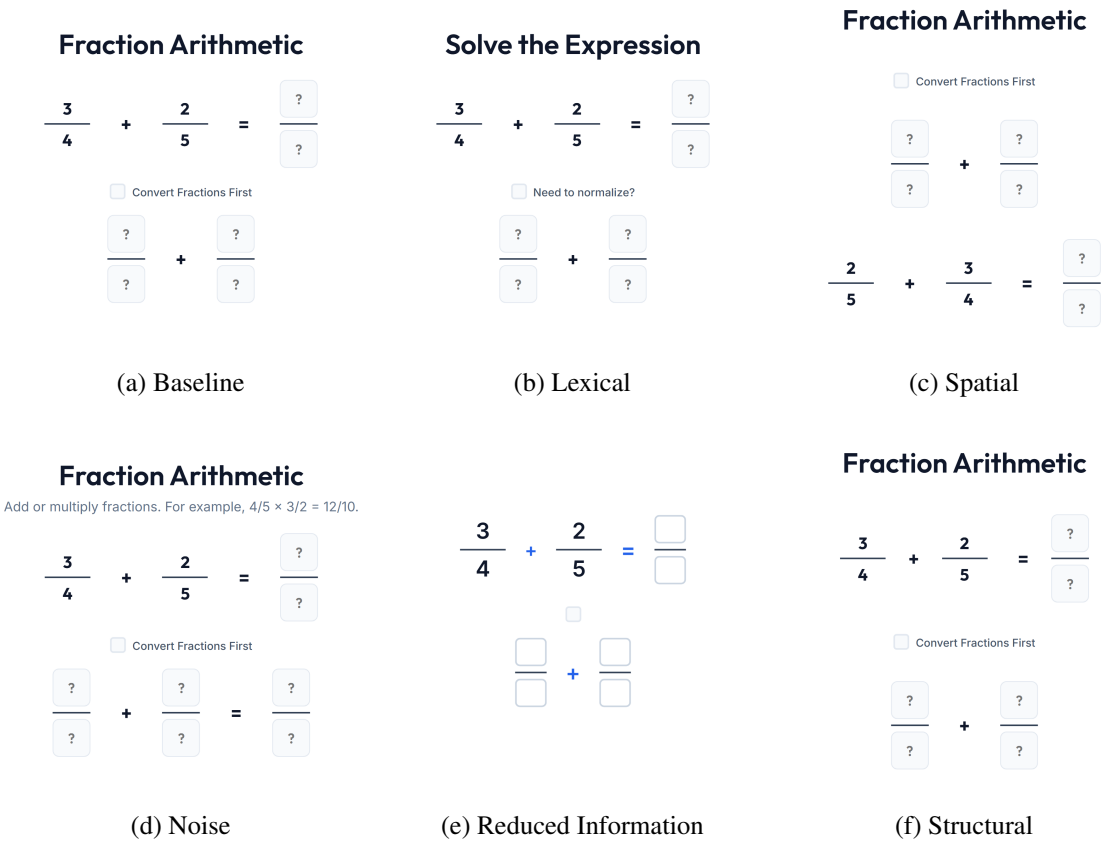
sessions per architecture and 1,500 sessions across the three architectures. The system and tutor were reset before every problem attempt.

4.3.2 Language Model Configuration

We used OpenAI's GPT-5.6 Luna model for both the L-HTN and LLM-agent evaluations. We selected this model for its fast inference speed and cost-efficiency. We configured the reasoning effort parameter to 'xhigh' to ensure the model could reason and produce quality outputs. The maximum number of output tokens was capped at 10,000 to accommodate each model's complex reasoning chains and the additional tokens consumed by LLMs when processing images. The model hyper-parameters were fixed at a temperature of 0.2 to help ensure deterministic and reproducible results.

4.3.3 Measures

The primary outcome is whole-problem success. A Fraction Arithmetic run succeeds when the final numerator and denominator and any required intermediate values are correct. A Prisoners and Guards



(a) Baseline (b) Lexical (c) Spatial

(d) Noise (e) Reduced Information (f) Structural

Figure 2: The six experimental interface conditions for the Fraction Arithmetic tutor. Note that the Structural condition appears visually identical to Baseline because its perturbations modify the hierarchical relationships and grouping of elements within the underlying HTML rather than in the interface.

run succeeds when all guards and prisoners reach the destination bank without violating the task constraints.

4.4 Study 2 Adapting to Environment Changes

4.4.1 Design and Procedure

Study 2 tests how robust each architectures is to changes in environment representation. We hold the underlying tasks and architectures fixed while evaluating five variations of the interfaces from study 1 in each domain. The five variations modify the interfaces along several dimensions: lexical, spatial, structural, task-irrelevant (or noise), reduced-information. Table 1 describes the specific manipulations applied to each interface, and Figure 2 shows how the six Fraction Arithmetic interfaces appear visually.

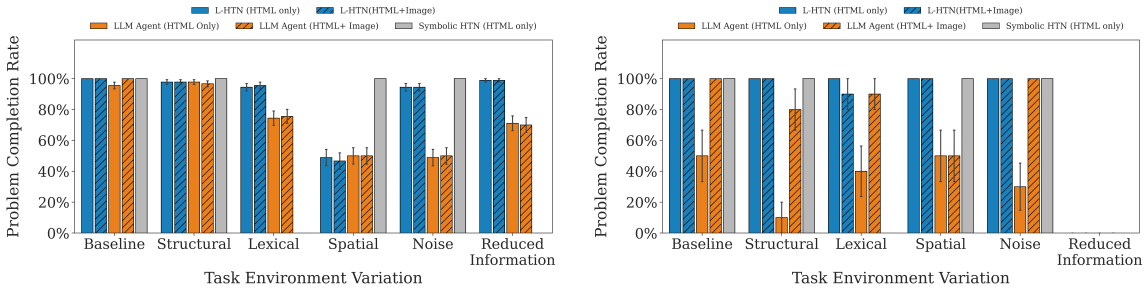


Figure 3: Comparison of problem completion rates for both domains, Fraction Arithmetic (left) and Prisoners and Guards (right). We report standard error.

4.4.2 Observation Conditions

The Linguistic HTN and LLM Agent architectures are evaluated with two forms of environment observation. In the *HTML-only* condition, the language model receives the current HTML as in study 1. In the *HTML-plus-Image* condition, it receives the same DOM together with a PNG image of the web interface.

The image helps determine spatial relationships, while the DOM gives the identifiers needed to reference and manipulate interface elements. In the HTML-plus-image condition, each DOM element also includes its bounding box in image coordinates. This links its HTML representation to its rendered location. This comparison tests whether visual information helps the language model use its available procedural knowledge when interpreting the modified states.

The Linguistic HTN and LLM Agent are each evaluated under all 12 combinations of interface variations and observation mode. The Symbolic HTN is evaluated once under each of the six interface configurations and operates only on the symbolic state derived from the DOM.

4.4.3 Measures and Analysis

Whole-problem success remains the primary outcome and follows the same domain-specific criteria as Study 1. We report success by architecture, domain, and interface configuration. For the Linguistic HTN and LLM Agent, we additionally compare success between the DOM-only and DOM-plus-image conditions.

For the two LLM-based architectures, we also report the mean number of LLM calls, input and output tokens, execution time, and estimated API cost per run. We calculate cost from provider-reported token usage and the pricing schedule in effect at the time of these experiments.

5. Results

Figure 3 compares problem completion rates for the three planning systems across the Fraction Arithmetic and Prisoners and Guards domains. Under the baseline interfaces, the symbolic HTN completed 100% of problems in both domains, while the L-HTN achieved near-perfect completion with both HTML-only and HTML-plus-image observations. The small number of L-HTN failures

in Fraction Arithmetic occurred when the language model returned an incorrect field identifier. For the LLM-agent, the largest difference appeared in Prisoners and Guards, where it completed approximately 50% of problems using HTML alone but 100% when the screenshot was also provided. In contrast, adding the screenshot produced little change in baseline performance for the L-HTN. The baseline results therefore show relatively similar performance between the two HTN planners, while the LLM-only agent's performance depended more strongly on the observation modality.

For several of the environment variations, including the structural, spatial, and noise conditions, the symbolic HTN completed 100% of problems in both domains, but its performance fell sharply when the variations removed or changed information used by its encoded representation. In both tasks, it completed 0% of problems under both the lexical and reduced-information conditions, while the L-HTN completed approximately 95% and 99% of those same problems, respectively. In the Fraction Arithmetic spatial condition, L-HTN performance fell to approximately 47–49% across the two observation settings, and the LLM-only agent completed approximately 50%, while the symbolic HTN remained at 100% in the same condition. Reduced information produced another boundary, but its effect differed by domain. The L-HTN remained near 99% in Fraction Arithmetic while the LLM-only agent completed roughly 70% and the symbolic HTN completed 0%. In Prisoners and Guards, all three systems completed 0% of problems under reduced information.

In general, providing screenshots did seem to have an effect on performance, but this effect was varied. L-HTN performance in Fraction Arithmetic was nearly unchanged between HTML-only and HTML-plus-image observations across most conditions. Prisoners and Guards showed the same general pattern, with the lexical condition providing the largest difference at 100% with HTML alone and approximately 90% with the screenshot. The LLM-only agent showed much larger changes in Prisoners and Guards. For example, performance increased from approximately 50% to 100% in the baseline condition, from 10% to 80% in the structural condition, and from 30% to 100% in the noise condition when the screenshot was added.

5.1 LLM Usage and Computational Cost

Across both domains and observation settings, the experiments incurred a total LLM API cost of $29.26. Runs that included image observations accounted for $18.88 of this cost, compared with $10.38 for runs using HTML alone. Image-based runs therefore cost approximately 82% more in aggregate than the corresponding text-only runs. In Fraction Arithmetic, including the screenshot increased the average LLM query time from 2.58 seconds with HTML alone to 2.74 seconds. In Prisoners and Guards, the corresponding averages were 2.88 seconds and 4.09 seconds, respectively. Thus, image observations increased the computational cost of the experiments but only produced a small increase in average processing time per LLM query.

6. Discussion

Our results provide initial evidence that linguistic HTNs can retain useful procedural knowledge across changes in environment representation. The planner solved the baseline tasks and, to varying degrees, continued to apply the same authored HTNs across the environment variations. Interestingly, the mixed effect of images showed that visual information sometimes improved performance,

particularly when spatial organization provided useful context, but it was not always beneficial. In some conditions, adding the image reduced performance. A image contributes information that may be difficult to recover from the DOM, but it also increases the amount of context the model must process. These findings suggest that choosing how much environment information to provide is itself an important design decision. Too little information can remove cues needed for task grounding, while too much can introduce noise or competing interpretations.

The planning architecture partly addresses this problem through constrained, local reasoning. At each decomposition step, the language model only receives the current task context and environment state, rather than the complete planning history. The HTN maintains the global procedural context, allowing the model to focus on a smaller, localized decision. This reduces prompt size and limits the model's choices, but it also raises open questions about context management. Some future tasks may require information from earlier parts of the plan that is not in the current task context, so propagating the right amount of relevant context is an important design consideration.

The architecture also suggests a more tightly coupled way of integrating language models with cognitive systems. In the planner, components of the HTN are mapped directly onto mechanisms provided by the language model. Tasks are represented as function tools, task arguments become function parameters, and decomposition proceeds through constrained function calls. This allows the language model to perform semantic interpretations through mechanisms it is already designed to use, while the HTN determines which tasks and decompositions are available at each point in the procedure. More generally, cognitive systems that heavily incorporate language models may benefit from identifying similar mappings between their computational structures and the native capabilities of LLMs, rather than treating the model as an independent reasoning component.

Overall, the language model's interpretive capabilities provide a lot of flexibility, but this can introduce several challenges. Because LLM decisions are not deterministic or guaranteed to be correct, the planner may occasionally select an inappropriate task or decomposition, although careful prompting and well-specified task descriptions can reduce these errors. There is also a tradeoff in how generally HTN knowledge should be expressed. For example, an early version of the Prisoners and Guards HTN described movement in terms of a 'source" and 'destination" bank, but these terms became ambiguous as the boat changed locations. More generally, linguistic HTN descriptions must be designed in such a way that only elements intended to be semantically mapped share similar meaning. This is because a poorly described HTN may lead the planner to incorrectly treat two task descriptions as equivalent or fail to recognize a valid relationship. Naming and description quality therefore become important parts of the representation. The Noise and Reduced Information conditions are an example of this tradeoff, as plausible distractions or ambiguities can leave insufficient information for planning to occur.

Other important considerations are computational and monetary cost. Because a language model is queried repeatedly during decomposition, latency accumulates over multi-step procedures. In our experiments, a language model query to instantiate a task with arguments lasted approximately 2-3 seconds on average, while the average time for the initial task segmentation step of the algorithm was at an average of approximately 7 seconds. Furthermore, adding images increased planning time by approximately $3.5x$, even though total API token cost remained low. Even though speed is a major factor in determining feasibility of the approach, it is not a hindrance for human-computer-

based interactive systems such as intelligent tutoring systems. Finally, performance also depends on the capabilities of the selected language model. A weaker model may make more grounding or applicability errors, while a stronger model may increase latency, resource requirements, or monetary cost. This is an additional trade-off that must be considered.

7. Related Works

7.1 Formal HTN Planning and Knowledge Acquisition

HTN domains encode how tasks decompose and the conditions under which those decompositions apply. When these descriptions are insufficient for a situation encountered during planning, prior systems have adapted their knowledge as needed. Xu & Munoz-Avila (2025), for example, learns generalized HTN methods from LLM-generated decompositions when no existing methods are applicable. L.E.A.R.N. (Smith & MacLellan, 2025) uses an LLM to generate solution traces to problems that serve as demonstration examples for an interactive task learning algorithm that constructs HTNs. Further, other systems adapt the representations that connect procedural knowledge to the current environment. For example, Our concern is complementary to these approaches. We seek to reduce the need to modify the underlying procedural knowledge when the same task is encountered in a differently represented environment. The Linguistic HTN Planner encodes its knowledge in English sentences and retains a structured hierarchy of problem-solving knowledge while using a language model to interpret the semantic relationship between its natural-language task and method descriptions and observations of the current environment.

Language has also been used as an interface through which procedural knowledge can be provided to cognitive systems. Langley et al. (2021) describes the acquisition of HTN procedures from written instructions through syntactic analysis and semantic interpretation. More recently, VAL uses LLM-based modules to interpret natural-language instructions, resolve these instructions to action predicates, and construct reusable symbolic hierarchical task knowledge (Lawley & Maclellan, 2024). Kirk et al. (2022) likewise examine LLMs as sources of task knowledge and explore how language models can be used to reconcile the task representation with the agent's interpretive capabilities.

These systems use language or LLMs to bridge natural language descriptions and other structured symbolic representations. The goal of our system slightly differs. Instead of interpreting a natural-language procedure once and converting it into a fixed relational representation, linguistic descriptions are retained within the HTN and interpreted as planning proceeds. Tasks still use a basic functional notation within the language model so that values can be grounded and propagated through the network, but the HTN remains linguistically specified.

Recent systems either use LLMs as planners or combine LLMs with more traditional symbolic planning mechanisms to obtain highly flexible, yet reliable performance. LLM-only planners can generate plausible decompositions of high-level tasks, but their plans may contain unavailable actions, omit necessary steps, or otherwise fail to be executable (Huang et al., 2022). STARS (Kirk et al., 2024) addresses related problems by having a cognitive agent analyze, repair, and select LLM-generated and verified task knowledge before it is used. These systems either use the language model

to assist in planning or help repair procedural knowledge that would otherwise be unavailable to the agent in real-time.

The Linguistic HTN Planner differs from these approaches in how the language model participates in the planning process. For example, rather than asking the model to generate a complete plan that is later checked or repaired, planning proceeds through a sequence of local decomposition decisions. At each step, the HTN maintains the current procedural context and determines what decisions remain, while the language model interprets the current state and selects among the locally available decomposition alternatives. This organization uses the LLM for the semantic judgments at which it is most useful while leaving long-horizon procedural control to the planning algorithm.

8. Future Work and Conclusion

This work introduced the Linguistic Hierarchical Task Network (L-HTN), which represents procedural knowledge in the form of natural language. Across two procedural tutoring tasks, the planner achieved near-perfect performance in the baseline environments and retained varying levels of performance when the same tasks were presented through alternate environment configurations. These results suggest that L-HTNs can reduce the representational dependence between HTNs and a particular environment. However, performance remains sensitive to the quality and amount of information provided about the current environment and how linguistic descriptions in the HTN are expressed.

Future work should examine these sensitivities more systematically. For example, additional experiments might more robustly vary the ambiguity of the HTN and environment descriptions, the amount of state information provided to the planner, and the planning context available during decomposition. In particular, selectively providing earlier planning results may improve task selection and grounding without substantially increasing noise in the prompt. Evaluating additional commercial and open language models would help determine how much performance depends on the semantic capabilities of the underlying model and clarify tradeoffs among accuracy, latency, and computational and monetary cost. Because repeated LLM calls also contribute considerable latency, another direction is to learn a more specific, symbolic representation from successful planning episodes so that the language model does not need to be queried as much. In this case, the linguistic structure serves as a template that can still be used in various environments out-of-the-box, but the system learns a representation for that environment to save on latency and computational cost.

More broadly, the Linguistic HTN Planner shows how we might better couple cognitive systems with the native capabilities of language models. By mapping HTN tasks onto function tools, the system maintains the strengths and reliability of both approaches. This kind of structural mapping may be a useful direction for integrating language models into other cognitive architectures.

References

Georgeff, M. P., & Lansky, A. L. (1987). Reactive reasoning and planning. *AAAI* (pp. 677–682).

Huang, W., Abbeel, P., Pathak, D., & Mordatch, I. (2022). Language models as zero-shot planners: Extracting actionable knowledge for embodied agents. *International conference on machine*

learning (pp. 9118–9147). PMLR.

Kambhampati, S., Valmeekam, K., Guan, L., Verma, M., Stechly, K., Bhambri, S., Saldyt, L., & Murthy, A. (2024). Llms can't plan, but can help planning in llm-modulo frameworks. *arXiv preprint arXiv:2402.01817*.

Kirk, J. R., Wray, R. E., Lindes, P., & Laird, J. E. (2022). Improving language model prompting in support of semi-autonomous task learning. *arXiv preprint arXiv:2209.07636*.

Kirk, J. R., Wray, R. E., Lindes, P., & Laird, J. E. (2024). Improving knowledge extraction from llms for task learning through agent analysis. *Proceedings of the AAAI Conference on Artificial Intelligence* (pp. 18390–18398).

Langley, P., Iba, W., & Shrager, J. (1994). Reactive and automatic behavior in plan execution. *AIPS* (pp. 299–304).

Langley, P., Shrobe, H., & Katz, B. (2021). Cognitive task analysis of rapid procedure acquisition from written instructions. *Proceedings of the Ninth Annual Conference on Advances in Cognitive Systems*.

Lawley, L., & Maclellan, C. (2024). Val: Interactive task learning with gpt dialog parsing. *Proceedings of the CHI Conference on Human Factors in Computing Systems*. New York, NY, USA: Association for Computing Machinery. From `https://doi.org/10.1145/3613904.3641915`.

Munoz-Avila, H., Aha, D. W., & Rizzo, P. (2025). Chathtn: Interleaving approximate (llm) and symbolic htn planning. From `https://arxiv.org/abs/2505.11814`.

Nau, D. S., Au, T.-C., Ilghami, O., Kuter, U., Murdock, J. W., Wu, D., & Yaman, F. (2003). Shop2: An htn planning system. *Journal of artificial intelligence research*, *20*, 379–404.

Nejati, N., Langley, P., & Konik, T. (2006). Learning hierarchical task networks by observation. *Proceedings of the 23rd international conference on Machine learning* (pp. 665–672).

Oruganti, S., Nirenburg, S., McShane, M., English, J., Roberts, M., Arndt, C., Parasuraman, R., & Sentis, L. (2026). Why cognitive robotics matters: Lessons from ontoagent and llm deployment in harmonic for safety-critical robot teaming. *arXiv preprint arXiv:2603.26730*.

Smith, G. R., & MacLellan, C. J. (2025). Learn: A hybrid architecture for language-guided induction of hierarchical task networks. *Twelfth Annual Conference on Advances in Cognitive Systems*.

Valmeekam, K., Olmo, A., Sreedharan, S., & Kambhampati, S. (2022). Large language models still can't plan (a benchmark for llms on planning and reasoning about change). *NeurIPS 2022 Foundation Models for Decision Making Workshop*.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, *30*.

Xu, Y., & Munoz-Avila, H. (2025). Online learning of htn methods for integrated llm-htn planning. *arXiv preprint arXiv:2511.12901*.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.

Zhao, W. X., et al. (2023). A survey of large language models. *arXiv preprint arXiv:2303.18223*, *1*.

Appendix A: Next Task Selection Prompt Template

Next Task Selection Prompt

```
--- REQUIREMENT: SELECT THE NEXT TASK IN THE PLAN ---

--- Begin Context and Instructions ---

Context: You are given:

1. The parent task currently being decomposed, including any arguments already bound for that task
.
2. A description of the current environment state.
3. A list of candidate methods that may decompose the parent task.

Each candidate method contains preconditions that determine when the method is applicable and an
ordered list of subtasks.

Your job is to find an applicable grounding of a candidate method and return a function tool call
for the FIRST subtask of that method.

A method is applicable only if there is a single, consistent set of argument bindings under which
ALL of its preconditions are satisfied by the current environment state.

Preconditions may serve two purposes:

* They may test whether already-bound arguments satisfy required conditions.
* They may bind previously unbound arguments by identifying values in the environment that satisfy
 the preconditions.

Bindings supplied by the parent task are fixed. Do not replace or reinterpret a parent-task
argument in order to make a method applicable. Preconditions may only verify those bindings or
introduce additional bindings for values that are not already bound.

Use semantic interpretation when relating natural-language preconditions to the environment state.
 Exact wording or syntactic correspondence is not required. However, semantic interpretation does
NOT permit relaxing a precondition: every precondition must actually be satisfied under the
selected grounding.

--- Method Evaluation Procedure ---

Evaluate every candidate method as follows:

1. Begin with all argument bindings inherited from the parent task.

2. Evaluate the candidate method's preconditions against the current environment state.

3. When a precondition refers to a value that is not already bound, search the environment state
for a value that can bind it.

4. Maintain one consistent set of bindings across the entire method. All preconditions must be
simultaneously true under the SAME grounding. Do not satisfy different preconditions using
incompatible assignments.

5. If no consistent grounding exists that satisfies every precondition, the method is NOT
applicable.

6. If at least one consistent grounding satisfies every precondition, the method is applicable.
Retain the bindings that established its applicability.

7. After evaluating all candidate methods, select an applicable method. If multiple methods are
applicable and no other selection priority is specified by the domain, select the first applicable
 method in the order provided.

8. Return a function tool call corresponding to the FIRST subtask of the selected method.

9. Ground the arguments of that function call using the SAME bindings that established the
selected method's applicability, together with any applicable parent-task arguments. Do not
independently re-ground values after selecting the method.

Subtasks may be expressed in natural language and therefore may need to be mapped to one of the
provided function tools. Use the provided tool names, descriptions, and argument schemas as the
authoritative definitions of the available tasks and their required arguments.

--- End Context and Instructions ---

--- Begin State Description ---
```

The code for the L-HTN system can be found here: `https://github.com/Teachable-AI-Lab/LHTN/`

```
{state_description}

--- End State Description ---

--- Important Rules ---

* Evaluate candidate methods according to the semantic meaning of their preconditions, not merely
lexical similarity between the preconditions and the state description.

* A method is applicable only when ALL of its preconditions are simultaneously satisfied under one
 consistent grounding.

* Preconditions may bind arguments that were not supplied by the parent task.

* Parent-task argument bindings are fixed and must be propagated wherever the corresponding value
is required. Do not overwrite a parent-task binding with a different value extracted from the
environment.

* Any additional argument value must be explicitly supported by the environment state or otherwise
 established while evaluating the selected method's preconditions. Do not fabricate values.

* The grounding used in the returned first-subtask function call must be consistent with the
grounding that made the selected method applicable.

* Return a function call for the FIRST subtask only. Do not execute, return, or reason ahead to
later subtasks.

* Set the parameter 'chosen_method_name' to the exact name of the selected candidate method (the
string following 'Method: ...'), NOT the name of the subtask or function tool call.

* The first subtask may be the same task as the parent task or another compound task. Do not
reject a subtask merely because it is identical or similar to the parent task.

* If the selected candidate method has no subtasks (e.g., 'Subtasks: None'), return a function
tool call for the current task ({current_task}) and set 'chosen_method_name' to the exact name of
the selected method.

* If NO candidate method has any grounding under which all of its preconditions are satisfied, do
not return a function call.

--- End Important Rules ---

--- Begin Parent Task ---

Task: {current_task}
Arguments: {task_args}

--- End Parent Task ---

--- Begin Candidate Methods ---

{candidate_methods}

--- End Candidate Methods ---

Return exactly one valid function tool call for the first subtask of the selected applicable
method, and nothing else.
```

Appendix B: Sample Prompts for LLM-Only Baseline and Function Tool Schemas

Fraction Arithmetic Prompt

Prompt:

```
Current Environment Observation: {state_observation}

Domain Information:
You are an autonomous planning solving a fraction arithmetic problem in a web interface. When
adding or subtracting fractions with different denominators, you should convert the fractions
first. When dividing, you should convert the fractions by taking the reciprocal of the second
fraction and performing a multiplication. Show your work if you need to convert the fractions.

Instructions:
1. Examine the current environment observation carefully.
2. Choose exactly ONE function tool call to execute next.
3. Do not attempt to execute multiple actions in a single step.
4. To get an updated state, you should call your get_state function only.
5. If the task is completely achieved and no further action is needed, return no tool calls to
terminate.
```

Prisoners and Guards Prompt

Prompt:

```
Current Environment Observation: {state_observation}

Domain Information:
You are an autonomous planning agent executing primitive actions in an environment. You are
solving the prisoners and guards river crossing problem in a web interface. Your objective is to
move all guards and prisoners safely from the left bank to the right bank. The boat can carry at
most 2 people and requires at least 1 person to row across. The boat can only depart from the bank
 where it is currently located. Crucially, at no point on either bank may prisoners outnumber
guards unless there are zero guards on that bank (if guards are present on a bank, the number of
guards must be greater than or equal to the number of prisoners).

Instructions:
1. Examine the current environment observation carefully.
2. Choose exactly ONE primitive tool call to execute next.
3. Do not attempt to execute multiple actions in a single step.
4. To get an updated state, you should call your get_state function only.
5. If the goal is completely achieved and no further action is needed, return no tool calls to
declare completion.
```

The code for the L-HTN system can be found here: https://github.com/Teachable-AI-Lab/LHTN/

Appendix C: Example Domain Tasks as LLM Function Tool Schemas

Fraction Arithmetic Root Task Schema

```
{
  "type": "function",
  "name": "
Perform_fraction_arithmetic_on_two_fractions
",
  "description": "This task fully solves
fraction addition, subtraction,
multiplication, and division problems. The
task has methods for solving fraction
multiplication and division problems. When
we have an addition or subtraction problem,
the task has methods of solving scenarios
where the denominators are the same and when
 the denominators are different and the
fractions need to be converted first.",
  "strict": true,
  "parameters": {
    "type": "object",
    "properties": {
      "fraction1": {
        "description": "The first fraction
to perform arithmetic on.",
        "type": "string"
      },
      "fraction2": {
        "description": "The second fraction
to perform arithmetic on.",
        "type": "string"
      }
    },
    "required": [
      "fraction1",
      "fraction2"
    ],
    "additionalProperties": false
  }
}
```

Prisoners and Guards Root Task Schema

```
{
  "type": "function",
  "name": "solve_river_crossing",
  "description": "Move all guards and
prisoners from the left bank to the right
bank safely using a 4-phase strategy:\n1.
Reduce prisoners on the left bank until
guards can safely move.\n2. Transfer all
guards to the right bank using the balanced
pivot maneuver.\n3. Sweep the remaining
prisoners across to the right bank.\n4.
Finalize and complete the problem.",
  "strict": true,
  "parameters": {
    "type": "object",
    "properties": {},
    "required": [],
    "additionalProperties": false
  }
}
```

The code for the L-HTN system can be found here: `https://github.com/Teachable-AI-Lab/LHTN/`